

COMSATS University Islamabad, Vehari Campus Pakistan

Department of Computer Science

Course: CSC336 – Web Technologies Assignment 3

GitHub Repository Link:

<https://github.com/ahsanalich358/Assignment3WebTechnologies>



**Building a Laravel-Based To-Do List Management System
with Full CRUD Functionality, Priority Management,
Status Tracking, and Filter Feature**

for the Degree of

Bachelor of Science in Software Engineering

Submitted By: Ahsan Ali
Student ID: CIIT/SP24-BSE-004/VHR
Instructor: Mam Yasmeen Jana
Semester: Spring 2026

Abstract

This report presents the design and implementation of a web-based To-Do List Management System developed using the Laravel PHP framework. The system is built following the Model–View–Controller (MVC) architectural pattern and provides a complete suite of CRUD (Create, Read, Update, Delete) operations for managing task records. Key features include task priority levels (Low, Medium, High), due date assignment, status tracking (Pending/Completed), filter by status and priority, and a real-time statistics dashboard. The application is deployed locally using Laravel’s built-in Artisan development server and uses MySQL as the backend database. The project demonstrates practical application of Laravel routing, Eloquent ORM, Blade templating, and form handling in a real-world web development scenario.

Keywords: Laravel, CRUD, MVC, PHP, To-Do List, Task Management, Priority, Status Tracking, Blade Templates, Eloquent ORM, MySQL.

List of Abbreviations

Abbreviation	Full Form
CRUD	Create, Read, Update, Delete
MVC	Model–View–Controller
PHP	Hypertext Preprocessor
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
SQL	Structured Query Language
URL	Uniform Resource Locator
ORM	Object Relational Mapping
IDE	Integrated Development Environment
UI	User Interface
DB	Database
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
CMD	Command Prompt / Terminal

Contents

Abstract	i
List of Abbreviations	ii
1 Introduction and System Overview	1
1.1 Background and Motivation	1
1.2 Tools and Technologies	1
1.3 Project Objectives	1
2 Implementation	3
2.1 Development Environment	3
2.2 Database Schema	3
2.3 Project Setup and Execution	3
2.4 Application Routes	4
2.5 Core Implementation Details	5
2.5.1 CRUD Operations	5
2.5.2 Task Model and Query Scopes	6
2.5.3 Key Features	6
2.5.4 Website Screenshots	7
Conclusion	10

List of Tables

1.1	Tools and Technologies Used	2
2.1	Development Environment Setup	3
2.2	Database Table: tasks	4
2.3	Application Routes	5

List of Figures

2.1	Main Dashboard – Task list with statistics and filter controls	7
2.2	Add New Task – Form with title, description, due date, and priority fields	8
2.3	Edit Task – Pre-populated form allowing update of all task fields	9
2.4	Filter Feature – Tasks filtered by priority or status using dropdown selectors	9
2.5	task completed	9

Chapter 1

Introduction and System Overview

1.1 Background and Motivation

Managing daily tasks and work items efficiently is a fundamental requirement in modern life — ranging from personal productivity tools and academic assignment trackers to professional project management systems. Traditional methods of task management, such as handwritten lists or spreadsheets, are inefficient, easy to lose, and difficult to scale when tasks grow in number.

This project, **Building a Laravel-Based To-Do List Management System with Full CRUD Functionality, Priority Management, Status Tracking, and Filter Feature**, is designed to address these challenges by implementing a complete task management solution using the Laravel framework. Laravel’s expressive syntax, built-in ORM (Eloquent), and Blade templating engine make it an ideal choice for rapid development of data-driven web applications.

The system enables users to add new tasks with a title, description, due date, and priority level; view all tasks in a structured table; update existing task details or change their status; and permanently delete tasks after explicit confirmation. A statistics dashboard shows real-time counts of total, pending, and completed tasks. Filter controls allow users to quickly narrow down tasks by status or priority level.

1.2 Tools and Technologies

Table 1.1 lists the tools and technologies selected for the development of this project. Each component was chosen based on compatibility, community support, and suitability for Laravel-based web development.

1.3 Project Objectives

The core objectives guiding the design and development of this system are:

- **PO-1:** Design and implement a fully functional web application that supports Create, Read, Update, and Delete operations on task records using the Laravel

Table 1.1: Tools and Technologies Used

Tool / Technology	Version	Purpose
Laravel	Latest (v11.x)	Primary PHP framework; implements MVC architecture and handles routing, ORM, and views.
PHP	8.x	Server-side scripting language required to execute Laravel applications.
Composer	Latest	PHP dependency manager used to install and manage Laravel packages.
MySQL	Latest	Relational database used to persist task records including title, priority, status, and due date.
Blade	Built-in	Laravel's templating engine used to build dynamic HTML views and forms.
Bootstrap 5	5.x	Frontend CSS framework used to create a responsive, clean user interface.
VS Code	Latest	Primary code editor used for writing and debugging the application.
Git & GitHub	Latest	Version control and remote repository hosting for the project codebase.

MVC framework.

- **PO-2:** Extend the base CRUD system with additional usability features – specifically, task priority assignment (Low, Medium, High), due date tracking, status toggling (Pending/Completed), filter by status and priority, and a real-time statistics dashboard.
- **PO-3:** Follow clean code and architectural conventions (MVC pattern, RESTful routing, Eloquent ORM) to produce a maintainable and extensible codebase.

Chapter 2

Implementation

2.1 Development Environment

The project was built and tested on a local Windows machine using the tools listed in Table 2.1.

Table 2.1: Development Environment Setup

Component	Tool / Technology	Role
Backend Framework	Laravel (v11.x)	MVC architecture, routing, ORM, validation.
Programming Lang.	PHP 8.x	Server-side execution of Laravel code.
Dependency Mgr.	Composer	Installs Laravel and all required packages.
Database	MySQL	Stores task records: title, description, due date, priority, status.
View Engine	Blade + Bootstrap 5	Dynamic UI pages: forms, tables, badges, alerts.
Code Editor	Visual Studio Code	Writing, debugging, and managing files.
Local Server	Laravel Artisan	Runs the application via <code>php artisan serve</code> .
Version Control	Git + GitHub	Tracks code changes and hosts the repository.

2.2 Database Schema

The application uses a single `tasks` table in MySQL. Table 2.2 describes its structure.

2.3 Project Setup and Execution

To run the project on a local machine, execute the following steps in the terminal from inside the project root directory:

Listing 2.1: Project Setup Commands

Table 2.2: Database Table: tasks

Column	Type	Description
id	BIGINT (PK, Auto)	Unique task identifier.
title	VARCHAR(255)	Short title of the task (required).
description	TEXT (nullable)	Optional detailed description of the task.
due_date	DATE (nullable)	The deadline date for the task.
priority	ENUM	Task priority: Low, Medium, or High.
status	ENUM	Task status: Pending or Completed.
created_at	TIMESTAMP	Auto-generated record creation timestamp.
updated_at	TIMESTAMP	Auto-generated last update timestamp.

```

1 # Step 1 - Create a new Laravel project
2 composer create-project laravel/laravel todo-app
3 cd todo-app
4
5 # Step 2 - Copy all provided project files into the project
   folder
6
7 # Step 3 - Configure the database in .env file
8 # Set: DB_DATABASE=todo_app DB_USERNAME=root DB_PASSWORD=
9
10 # Step 4 - Generate application key
11 php artisan key:generate
12
13 # Step 5 - Run database migrations to create the tasks table
14 php artisan migrate
15
16 # Step 6 - Start the local development server
17 php artisan serve

```

After the server starts, open `http://127.0.0.1:8000` in any browser to access the application.

2.4 Application Routes

Table 2.3 describes all the routes defined in `routes/web.php` for the To-Do List application.

Table 2.3: Application Routes

Method	URI	Controller Method	Route Name	Purpose
GET	/	index	tasks.index	Show all tasks with filters.
POST	/tasks	store	tasks.store	Save a new task to DB.
GET	/tasks/{task}/edit	edit	tasks.edit	Show the edit form.
PUT	/tasks/{task}	update	tasks.update	Update an existing task.
PATCH	/tasks/{task}/toggle	toggleStatus	tasks.toggle	Toggle Pending/-Completed.
DELETE	/tasks/{task}	destroy	tasks.destroy	Delete a task.
GET	/about	about	about	Show the About page.

2.5 Core Implementation Details

2.5.1 CRUD Operations

All CRUD operations are implemented through Laravel’s routing and a dedicated **TaskController**. The key controller methods are:

- **index():** Retrieves all task records (with optional status and priority filters) and passes them along with summary statistics (total, pending, completed counts) to the `tasks/index` Blade view.
- **store():** Validates incoming form data (title required, priority must be Low/Medium/High, due date must be valid date format) and persists a new task record to the `tasks` table with a default status of *Pending*.
- **edit(\$task):** Retrieves the specified task using Laravel’s route model binding and returns the pre-populated edit form view.
- **update(\$task):** Validates and saves changes to the selected task record, including any status change made via the edit form.
- **toggleStatus(\$task):** A dedicated PATCH route that flips the task status between *Pending* and *Completed* in a single click without opening the edit form.

- **destroy(\$task):** Permanently deletes the task record from the database after a JavaScript confirmation dialog prevents accidental deletion.

2.5.2 Task Model and Query Scopes

The Task Eloquent model defines the `$fillable` array for mass assignment protection and two local query scopes:

Listing 2.2: Task Model Scopes

```
1 // Filter by status
2 public function scopeByStatus($query, $status)
3 {
4     if ($status && $status !== 'All') {
5         return $query->where('status', $status);
6     }
7     return $query;
8 }
9
10 // Filter by priority
11 public function scopeByPriority($query, $priority)
12 {
13     if ($priority && $priority !== 'All') {
14         return $query->where('priority', $priority);
15     }
16     return $query;
17 }
```

These scopes allow clean, chainable filtering in the controller: `Task::byStatus($filter)->byPriority`

2.5.3 Key Features

- **Statistics Dashboard:** Three stat cards display real-time counts of Total, Pending, and Completed tasks at the top of the main page.
- **Priority Badges:** Color-coded badges (High = red, Medium = yellow, Low = green) make task urgency immediately visible in the task table.
- **Status Toggle:** A dedicated toggle button on each table row allows users to mark tasks Pending or Completed without navigating to the edit page.
- **Filter Controls:** Two dropdown selectors above the task table allow filtering by Status (All / Pending / Completed) and Priority (All / High / Medium / Low). Filters auto-submit on change via JavaScript.

- **Input Validation:** Server-side validation ensures the title is present, priority is a valid enum value, and due date is a valid date. Validation errors are displayed inline below each field.
- **Flash Messages:** Success notifications appear at the top of the page after every create, update, or delete operation and automatically dismiss after a few seconds.
- **Confirmation Dialog:** A JavaScript `confirm()` dialog prompts the user before any delete operation, preventing accidental data loss.

2.5.4 Website Screenshots

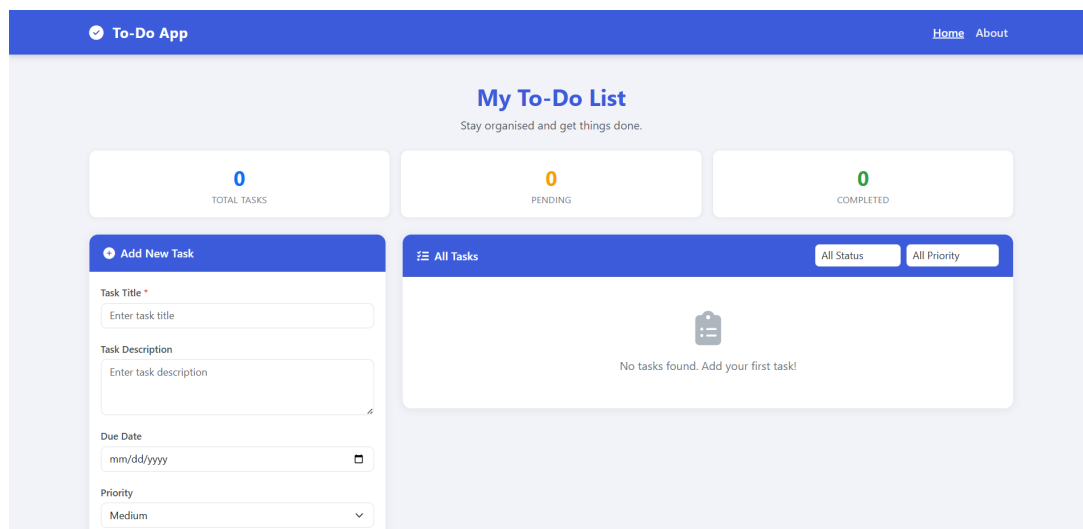
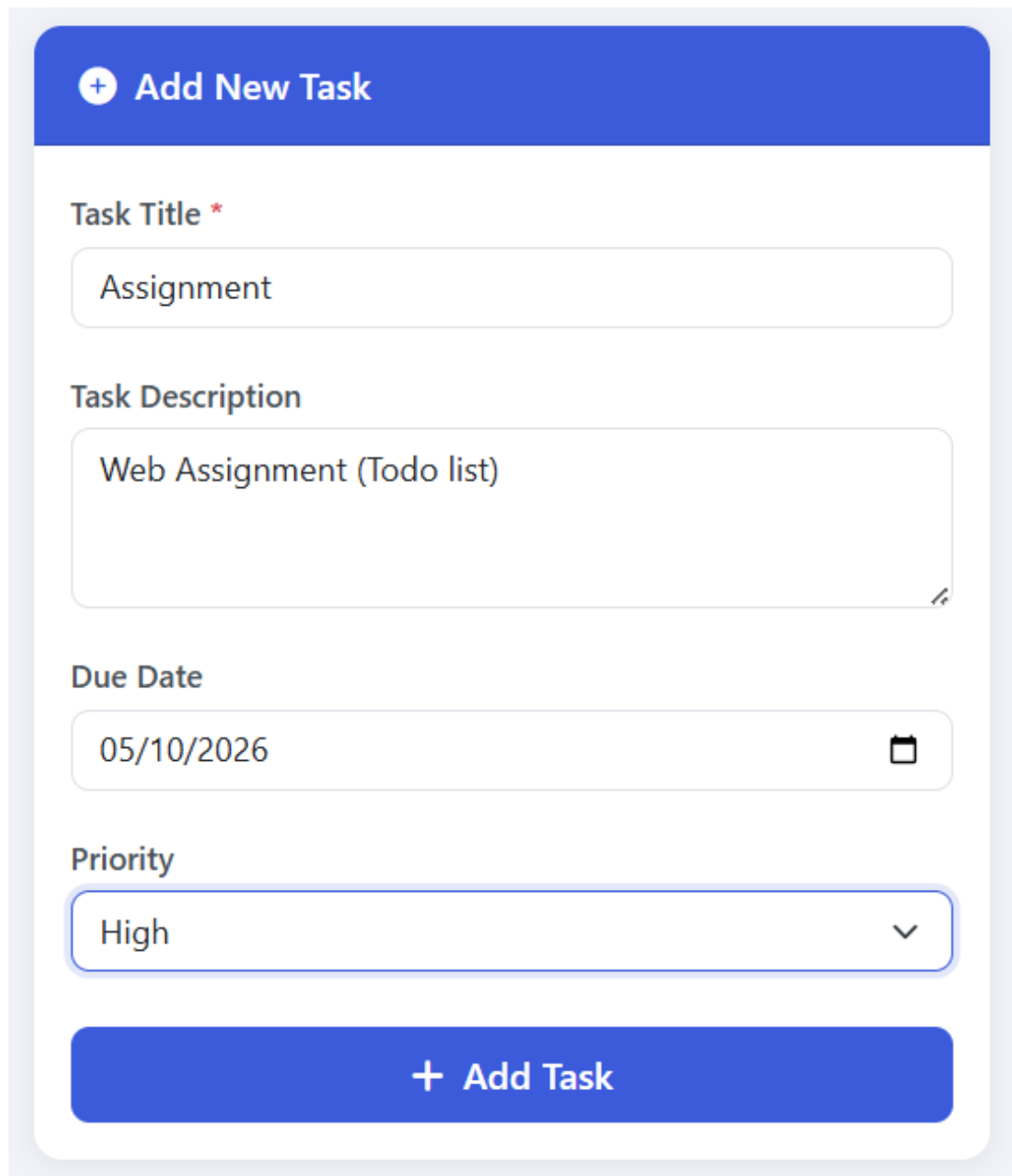


Figure 2.1: Main Dashboard – Task list with statistics and filter controls



The image shows a web form titled "Add New Task" with a blue header bar. The form contains four input fields: "Task Title" (with a red asterisk), "Task Description", "Due Date", and "Priority". The "Task Title" field contains the text "Assignment". The "Task Description" field contains the text "Web Assignment (Todo list)". The "Due Date" field contains the date "05/10/2026" and has a calendar icon. The "Priority" field is a dropdown menu showing "High" with a downward arrow. At the bottom of the form is a blue button with a white plus icon and the text "Add Task".

+ Add New Task

Task Title *

Assignment

Task Description

Web Assignment (Todo list)

Due Date

05/10/2026

Priority

High

+ Add Task

Figure 2.2: Add New Task – Form with title, description, due date, and priority fields

To-Do App Home About

← Back to Task List

Edit Task

Task Title *
Assignment

Task Description
Web Assignment (Todo list)

Due Date
05/10/2026

Priority
High

Status
Pending

Update Task Cancel

Figure 2.3: Edit Task – Pre-populated form allowing update of all task fields

My To-Do List
Stay organised and get things done.

1 PENDING 1 COMPLETED

All Tasks Pending All Priority

#	TITLE	DUE DATE	PRIORITY	STATUS	ACTIONS
1	Ai Assignment	12 May 2026	Medium	Pending	✓ ✎ 🗑

Figure 2.4: Filter Feature – Tasks filtered by priority or status using dropdown selectors

To-Do App Home About

Task status updated!

My To-Do List
Stay organised and get things done.

2 TOTAL TASKS 1 PENDING 1 COMPLETED

+ Add New Task

Task Title *
Enter task title

Task Description
Enter task description

All Tasks All Status All Priority

#	TITLE	DUE DATE	PRIORITY	STATUS	ACTIONS
1	Assignment Web-Assignment (Todo-list)	10 May 2026	High	Completed	↺ ✎ 🗑
2	Ai Assignment	12 May 2026	Medium	Pending	✓ ✎ 🗑

Figure 2.5: task completed

Conclusion

This assignment successfully demonstrates the practical implementation of a web-based To-Do List Management System using the Laravel PHP framework. The application covers all four CRUD operations in a clean, maintainable codebase structured according to the MVC design pattern. Additional features – task priority levels, due date tracking, status toggling, filter by status and priority, a real-time statistics dashboard, server-side input validation, and JavaScript confirmation dialogs – extend the system beyond a basic data-entry tool into a usable, real-world task management application.

Key learning outcomes achieved through this project include:

- Hands-on experience with Laravel routing, controllers, models, and Blade views.
- Practical understanding of Eloquent ORM for database interactions and local query scopes for clean filtering logic.
- Implementation of RESTful routing conventions including GET, POST, PUT, PATCH, and DELETE HTTP methods.
- Application of server-side validation and flash messaging for user feedback.
- Use of Blade templating features including `@foreach`, `@error`, `@csrf`, `@method`, and layout inheritance with `@extends` and `@yield`.
- Use of Git and GitHub for version control throughout the development cycle.

The project can be extended in future work by adding user authentication, role-based access control, task categories or tags, pagination for large task lists, due date reminder notifications, export to CSV/PDF, and deployment to a cloud hosting platform.

Bibliography

- [1] Laravel LLC. (2024). *Laravel Documentation – The PHP Framework for Web Artists*. Retrieved from <https://laravel.com/docs>
- [2] The PHP Group. (2024). *PHP Manual*. Retrieved from <https://www.php.net/manual/en/>
- [3] The Bootstrap Authors. (2024). *Bootstrap 5 Documentation*. Retrieved from <https://getbootstrap.com/docs/5.0/>
- [4] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.